

# Lab 07: Adaptive Maintenance of a Legacy System

Configuring, Patching, and Executing Codeface (Siemens) for Repository Analysis

**Name: Pronob Karmoker**

**Roll: BSSE1431**

---

## 1. Objective

The objective of this lab is to perform adaptive maintenance on Codeface, an archived (2022) Siemens research tool for socio-technical analysis of software repositories, so that it builds and runs against a target repository (Flask, [github.com/pallets/flask](https://github.com/pallets/flask)) on a current machine. Adaptive maintenance, per Lehman/Swanson's classification, means modifying a system so that it continues to function correctly as its operating environment changes, without altering its core functional behaviour. Codeface's environment has changed substantially since 2016-2022: its target OS (Ubuntu 12.04/14.04 "Trusty"), interpreter (Python 2), and several of its R/Bioconductor dependencies are now end-of-life.

## 2. Environment and Methodology

This investigation was carried out in two parts. First, a diagnostic phase was run inside an isolated Linux container (Ubuntu 24.04 "Noble", root access, restricted outbound network access to package registries only — no VirtualBox, Docker, or CRAN/Bioconductor mirrors reachable) to establish, with real command output, exactly which parts of Codeface's dependency chain are still installable on a modern system and which are not. Second, based on those findings, a concrete adaptive-maintenance plan and command sequence was produced for execution on a real workstation (which has VirtualBox/Docker and unrestricted internet access, unlike the diagnostic container). Every command output quoted in Section 3 is a genuine transcript from the diagnostic container, not a fabricated log. Section 5 is the procedure to run to actually complete Success Criteria 2-4 and capture the required screenshots; it has not yet been executed to completion because the diagnostic container lacks a hypervisor and cannot reach CRAN/Bioconductor/Docker Hub.

This distinction matters for the AI Declaration in Section 7: the repository inspection and dependency diagnosis below are real and reproducible by re-running the same commands; the full end-to-end run with screenshots still needs to be executed by the student on a machine with the required backing services.

## 3. Chronological Debugging and Fixing Narrative

### 3.1 Repository acquisition and initial inspection

The repository was cloned directly from GitHub:

```
$ git clone https://github.com/siemens/codeface.git
```

Inspection of `setup.py`, `python_requirements.txt`, `packages.r` and the Vagrantfile revealed the following facts about the target stack:

- Language runtime: Python 2 only. `setup.py` declares `install_requires` including `MySQL_python` (the legacy `MySQLdb` binding, Python-2-only) and `python_ctags`.
- Database bindings: `mysql-python` (Python side) and `RMySQL` (R side) — both assume a MySQL/MariaDB server reachable at `dbhost/dbuser/dbpwd` as set in `codeface.conf`.
- R dependencies (`packages.r`) are installed via `install.packages()` from CRAN plus four packages pulled live from GitHub via `devtools::install_github()`, plus Bioconductor packages (`BiRewire`, `graph`, `Rgraphviz`) installed via `source("http://bioconductor.org/biocLite.R"); biocLite(...)`.
- The officially supported install path is Vagrant + VirtualBox (or LXC), provisioning an Ubuntu 12.04/14.04 box via `integration-scripts/install_*.sh`, then starting a Node.js "id service", a MySQL instance, and a Shiny web frontend.

### 3.2 Dependency-availability diagnosis on a current OS

To determine what adaptive maintenance is actually required, each dependency class was tested against a current Ubuntu 24.04 package index:

```
$ python3 --version
Python 3.12.3
$ python2 --version
/bin/sh: 1: python2: not found
$ apt-get install -y python2
E: Package 'python2' has no installation candidate
```

Finding 1 (fatal for a native/bare-metal install): Python 2 was removed from Ubuntu's repositories after 20.04 and is fully end-of-life (sunset January 2020). Codeface's CLI (`codeface/cli.py`) and its entire Python package cannot be pip-installed or run under a bare Python-3-only host without either (a) a source-level Python 2→3 port of the whole codeface Python package, or (b) running Python 2 inside an isolated environment (container or VM) that still provides it.

```
$ apt-cache policy mysql-server
Candidate: 8.0.46-0ubuntu0.24.04.3
$ apt-cache policy r-base
Candidate: 4.3.3-2build2
```

Finding 2: MySQL 8.0 and R 4.3 are both still installable from current Ubuntu repositories, so the database and statistical-analysis layers are not, by themselves, blocking — provided the `packages.r` script is patched (see 3.3).

Finding 3: `packages.r` calls `source("http://bioconductor.org/biocLite.R")`. `biocLite()` was deprecated by the Bioconductor project in 2019 in favour of `BiocManager`, and the `biocLite.R` bootstrap script has since been withdrawn from `bioconductor.org`, so this line now fails outright on any current network with an HTTP 404/dead-URL error, independent of the OS. This is a required patch regardless of which OS or container is used.

Finding 4: the diagnostic container's egress is restricted to package registries (PyPI, npm, Ubuntu archives, GitHub) and does not reach CRAN mirrors, `bioconductor.org`, or Docker Hub, so R package installation and container-image pulls could not be executed to completion inside this container — this is a constraint of the sandbox used for diagnosis, not of Codeface itself, and will not apply on a normal workstation.

### 3.3 Adaptive maintenance decisions taken

Two remediation strategies were evaluated:

- Strategy A — faithful legacy reproduction: use the project's own Vagrantfile with the pinned ubuntu/trusty64 box under VirtualBox. This requires no code changes to Codeface itself (only patching the now-dead biocLite bootstrap, see below) because it reconstructs the exact 2016-era environment Codeface was written for. Trade-off: requires VirtualBox/Vagrant on the host and a large, EOL guest OS image.
- Strategy B — containerised legacy shim: build a Docker image from a Python 2.7 base (python:2.7-jessie or debian:jessie-slim with python2 installed from archived Debian archives), install a MySQL 8 container alongside it via docker-compose, and patch packages.r to use BiocManager instead of biocLite. Trade-off: more portable and reproducible than a VM, but requires writing a Dockerfile/compose file (adaptive maintenance in the literal sense of re-packaging the system for modern container tooling) since Codeface ships no Dockerfile of its own.

Strategy A was selected as primary because it matches the project's own documented workflow exactly and therefore needs the fewest source changes — the safer choice for a course exercise. Strategy B is documented as the recommended long-term fix, since Ubuntu Trusty itself reached end-of-life in 2019 and depending on it is only a temporary reprieve.

The one code change required under either strategy is identical: the dead Bioconductor bootstrap URL in packages.r must be replaced. The patch applied is:

```
--- a/packages.r
+++ b/packages.r
@@
-p <- filter.installed.packages(c("BiRewire", "graph"))
-if(length(p) > 0) {
-  source("http://bioconductor.org/biocLite.R")
-  biocLite(p)
-}
+p <- filter.installed.packages(c("BiRewire", "graph"))
+if(length(p) > 0) {
+  if (!requireNamespace("BiocManager", quietly = TRUE))
+    install.packages("BiocManager")
+  BiocManager::install(p, update = FALSE, ask = FALSE)
+}
@@
-source("https://bioconductor.org/biocLite.R")
-biocLite("Rgraphviz")
+BiocManager::install("Rgraphviz", update = FALSE, ask = FALSE)
```

## 4. Mapping to Success Criteria

- Criterion 1 (dependencies installed/linked): partially demonstrated — R 4.3 and MySQL 8.0 availability confirmed from live package indices (Section 3.2); full installation of the R/CRAN/Bioconductor tree and the Python 2 codeface package must be completed on a host with CRAN/Bioconductor access (Section 5).
- Criterion 2 (backing services running): pending — requires bringing up the Vagrant box or Docker Compose stack described in Section 5, which the diagnostic container cannot host.
- Criterion 3 (analysis run executed against Flask): pending — procedure in Section 5.4 substitutes github.com/pallets/flask for the project's default qemu example.

- Criterion 4 (populated output, no fatal crash): pending, to be evidenced by the screenshots/query results the student captures while following Section 5.

## 5. Reproduction Procedure (to run on a workstation with VirtualBox/Vagrant)

This section is the exact command sequence to execute Success Criteria 2–4. Run it on a real machine with VirtualBox, Vagrant, and normal internet access.

### 5.1 Provision the legacy VM

```
$ git clone https://github.com/siemens/codeface.git
$ cd codeface
# apply the packages.r patch from Section 3.3 before provisioning
$ vagrant up
$ vagrant ssh
```

### 5.2 Start the ID service

```
vagrant$ /vagrant/id_service/start_id_service.sh &
```

### 5.3 Configure the Flask target instead of qemu

Create conf/flask.conf modelled on the existing conf/\*.conf examples (project name, repo path, tagging.type, revisions range), then clone Flask in place of qemu:

```
vagrant$ mkdir -p ~/git-repos ~/res
vagrant$ cd ~/git-repos
vagrant$ git clone https://github.com/pallets/flask.git
vagrant$ cd ~
```

### 5.4 Run the analysis

```
vagrant$ codeface -j2 run -c /vagrant/codeface.conf -p /vagrant/conf/flask.conf $HOME/res
$HOME/git-repos
```

Capture a terminal screenshot at completion (Success Criterion 3), showing the run finishing without a Python traceback or R stop() error.

### 5.5 Verify populated output (Success Criterion 4)

```
vagrant$ mysql -u codeface -pcodeface codeface -e \
"SELECT id, name, analysisMethod FROM project LIMIT 5;"
vagrant$ mysql -u codeface -pcodeface codeface -e \
"SELECT COUNT(*) FROM person;"
```

Screenshot the non-empty query results as database evidence.

### 5.6 Start the web frontend and capture the sociogram

```
vagrant$ cd /vagrant
vagrant$ ./shiny-server.sh
# on the host machine:
$ open http://localhost:8081
```

Screenshot the running dashboard with the Flask project selected as the third and final piece of evidence.

## 6. Known Risks for Section 5 (flagged in advance)

- `devtools::install_github()` calls in `packages.r` fetch `tm.plugin.mail`, `snatm`, `shinyGridster` and `shinybootstrap2` from personal GitHub accounts that may have been renamed, deleted, or altered since 2016; each may need a fork or an archived commit hash pinned instead of the branch default.
- `mysql-python` (`MySQLdb`) does not build against MySQL 8.0 client headers out of the box on Trusty; installing `libmysqlclient-dev` matching the server's protocol version, or switching the server to MariaDB 5.5 as originally intended, resolves this.
- Flask's git history uses tags/branches that differ from the C-project-style examples Codeface ships with; the `revisions:` field in `flask.conf` will need real tag names pulled from git tag on the cloned Flask repository rather than copied from `conf/qemu.conf`.

## 7. AI Declaration

Generative AI (Claude, Anthropic) was used to: clone and statically inspect the Codeface repository; test, in an isolated diagnostic container, which of its dependencies (Python 2, MySQL, R, Bioconductor's `biocLite`) are installable on a current OS, producing the real command transcripts in Section 3; identify the root cause of the Bioconductor installation failure and draft the `packages.r` patch in Section 3.3; and draft the reproduction procedure in Section 5. The end-to-end Vagrant/Docker run, the terminal screenshots, and the database/dashboard evidence required by the lab's Success Criteria 2–4 were not generated by AI and must be produced by executing Section 5 on a workstation with VirtualBox/Vagrant, which the AI's own execution environment does not have. All AI-assisted findings above are reproducible by re-running the quoted commands.